

Lec5 StringStream

String Notes

1. String Literal

- string literal (字符串字面量) 本质上是 `const char`
- `std::string` 相当于可以动态管理 `const char` / `const char*`

```
const char* p = "Hello";  
p[0] = 'M'; // 报错，这里 p 指向只读内存  
  
std::string s2 = "Hello";  
s2[0] = 'M';  
s2 += "World"; // 都合法
```

C++

```
uint8_t arr[8] = {0};  
uint8_t* p = arr;  
  
// 此时 p = arr  
// *arr = *p  
// 可以理解为 arr 退化为指向数组第一个元素的指针
```

C++

2. String Comparison

- 可以用 `+` / `-` / `=` 来比较 string (按照字母表顺序)

3. 构造 String

```
string str1 = "Harry" + "Potter"; // 不合法  
string str2 = "Harry" + str;      // 合法
```

C++

4. String 成员函数

4.1 `str.size()` / `str.length()`

- 返回 `size_t`

4.2 `str.substr(i)` / `str.substr(i, n)`

- `n` 是 length

4.3 `str.compare()`

Table 24. Command Definitions

COMMAND	DESCRIPTION	1ST COMMAND BYTE	2ND COMMAND BYTE
WAKEUP	Completes SYNC and Exits Standby Mode	0000 0000 (00h)	
RDATA	Read Data	0000 0001 (01h)	
RDATAc	Read Data Continuously	0000 0011 (03h)	
SDATAc	Stop Read Data Continuously	0000 1111 (0Fh)	
RREG	Read from REG <i>rrr</i>	0001 <i>rrrr</i> (1xh)	0000 <i>nnnn</i>
WREG	Write to REG <i>rrr</i>	0101 <i>rrrr</i> (5xh)	0000 <i>nnnn</i>
SELFcal	Offset and Gain Self-Calibration	1111 0000 (F0h)	
SELFcal	Offset Self-Calibration	1111 0001 (F1h)	
SELFcal	Gain Self-Calibration	1111 0010 (F2h)	
SYScal	System Offset Calibration	1111 0011 (F3h)	
SYScal	System Gain Calibration	1111 0100 (F4h)	
SYNC	Synchronize the A/D Conversion	1111 1100 (FCh)	
STANDBY	Begin Standby Mode	1111 1101 (FDh)	
RESET	Reset to Power-Up Values	1111 1110 (FEh)	
WAKEUP	Completes SYNC and Exits Standby Mode	1111 1111 (FFh)	

NOTE: *n* = number of registers to be read/written – 1. For example, to read/write three registers, set *nnnn* = 2 (0010).
r = starting register address for read/write commands

4.4 `str.find()`

- 找一个字符串
- 找到：返回第一个字母的 index
- 没找到：返回 `std::string::npos`

4.5 `str.find_first_of()`

- 找 char

4.6 `str.replace()`

5. Parameter

5.1 常见写法

```
const std::string& str
```

5.2 如果参数是 `const std::string& str`，可以传入：

- `string`
- `string.substr(...)`
- `char*` / `const char*`
 - 会发生隐式转换：`string(const char*)`

5.3 str.c_str()

- 转换为 `const char*`

5.4 string_view str

- 只是看，不分配内存

Markdown

Stream Notes

1. Stream 是什么

- stream 可以理解成“数据流通道”
- `\`cin\`` / `\`cout\`` 是 console stream
- 文件也可以当 stream
- 字符串也可以当 stream

2. File Stream

2.1 `\`ifstream\``

- input file stream
- 用来从文件读数据
- 用法很像 `\`cin\``

```
```\cpp
```

```
#include <fstream>
using namespace std;
```

```
ifstream fin("test.txt");
```

```
int x;
string line;
```

```
fin >> x;
getline(fin, line);
```

## 2.2 ofstream

- output file stream
- 用来往文件写数据
- 用法很像 `cout`

C++

```
#include <fstream>
using namespace std;
```

```
ofstream fout("test.txt");
fout << 123 << " " << 4.5;
fout.close();
```

## 2.3 fstream

- 既能读也能写

```
fstream file("data.txt", ios::in | ios::out);
```

C++

## 2.4 is\_open()

- 检查文件有没有成功打开

```
if (file.is_open()) {
 // success
}
```

C++

# 3. Stream 作为条件

- stream 可以放进 `while (...)`
- 读取成功就是真
- 读取失败 / 到 EOF 就是假

```
while (fin >> x) {
 // 每次成功读到一个 x 就继续
}

while (getline(fin, line)) {
 // 每次成功读到一整行就继续
}
```

C++

# 4. String Stream

## 4.1 三种

- `istringstream`

- 从 string 里读
- `ostringstream`
  - 往 string 里写
- `stringstream`
  - 既能读也能写

```
#include <sstream>
using namespace std;

istringstream iss("123 45.6");
ostringstream oss;
stringstream ss;
```

C++

## 5. 常见用途

### 5.1 string -> number

```
string s = "123";
istringstream iss(s);

int x;
iss >> x;
```

C++

### 5.2 number -> string

```
int x = 123;
ostringstream oss;
oss << x;

string s = oss.str();
```

C++

### 5.3 解析一整行

- 先 `getline(...)`
- 再用 `istringstream` 继续拆这一行

C++

```
string line = "21:30";
istringstream iss(line);

int hour, minute;
char colon;

iss >> hour >> colon >> minute;
```

## 6. 常见操作

### 6.1 >>

- 从 stream 中提取数据
- 默认跳过空白
- 适合读单词、数字、字符

C++

```
istringstream iss("Jan 23 1955");
string month;
int day, year;

iss >> month >> day >> year;
```

### 6.2 <<

- 往 stream 里写数据

C++

```
ostringstream oss;
oss << 23 << " Jan " << 1955;
```

### 6.3 str()

- 取出 stream 当前内容，变成 `string`

C++

```
ostringstream oss;
oss << 123;
string s = oss.str();
```

## 6.4 peek()

- 偷看下一个字符
- 不取走

```
if (iss.peek() == ':') {
 // 下一个字符是 :
}
```

C++

## 6.5 get()

- 真正取走一个字符

```
char ch = iss.get();
```

C++

- `peek()`: 看, 不拿
- `get()`: 拿走

## 7. >> 和 getline 的区别

- `>>`
  - 按空白分隔
- `getline`
  - 读一整行

```
string s1, s2;
cin >> s1; // 只读到空格前
getline(cin, s2); // 读整行
```

C++

## 8. 随机访问 (不一定重点)

### 8.1 tellg() / tellp()

- `tellg()`
  - 当前读位置
- `tellp()`
  - 当前写位置

## 8.2 seekg() / seekp()

- seekg(...)
  - 移动读位置
- seekp(...)
  - 移动写位置

```
ss.seekg(0, ios::beg);
ss.seekp(0, ios::end);
```

C++

## 8.3 write()

- 原样写入指定数量的字符

```
stringstream ss;
ss.write("1234567890", 10);
```

C++

# 9. 最常记的模板

## 9.1 文件逐行读取

```
ifstream fin("data.txt");
string line;

while (getline(fin, line)) {
 // process line
}
```

C++

## 9.2 string 转 int

```
int stringToInt(const string& s) {
 istringstream iss(s);
 int x;
 iss >> x;
 return x;
}
```

C++



## 9.3 int 转 string

```
string intToString(int x) {
 ostringstream oss;
 oss << x;
 return oss.str();
}
```

C++

## 10. 易错点

- `ifstream` 是读文件，`ofstream` 是写文件
- `istream` 是从 string 读，`ostream` 是往 string 写
- `>>` 不是读整行，`getline` 才是
- `peek()` 不会把字符取走，`get()` 会
- `oss` 写完以后，别忘了 `oss.str()`